

## DATAKERN | Assignment 3

# The Financial Advisor Microservice

Context Injection • FastAPI • LLMs • Observability • Docker • Kubernetes

By Uma Kiran | DataKern | [www.data-kern.com](http://www.data-kern.com)

**Engineering mindset: Don't focus only on getting the answer. Focus on understanding what the application needs, where that information comes from, how the components connect, and how you can prove the system is working.**

DATAKERN

# 1. Before You Start

**Important learning rule: Do not use AI code-generation or coding-agent tools to produce your assignment solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use official documentation and search for errors, but you must write and explain your own solution.**

This assignment is designed to make you think like an AI Engineer. You are not expected to know every library API from memory. You are expected to understand the problem, make sensible engineering decisions, test your work, and use documentation when you need it.

# 2. The End Goal: What Are You Building?

Imagine that Apex Financial Partners has a mobile application. A client can ask:

**“I want to buy TSLA. Given my portfolio and the company risk policy, is this allowed? Also, is there any important news about TSLA today?”**

Your backend will provide the intelligence behind that experience.

- Interface: FastAPI REST endpoint `POST /chat`
- Intelligence: an LLM receives the question together with relevant context from PostgreSQL, the policy document, and live web news.
- Observability: Arize Phoenix captures the LLM interaction.
- Packaging: Docker.
- Deployment: local Kubernetes using minikube or kind.

This is a production-oriented learning prototype. It is intentionally simplified so you can focus on the engineering concepts.

# 3. What You Will Learn

By completing this assignment, you should be able to explain and demonstrate how an AI application can:

- retrieve structured context from a database
- extract relevant information from a large document
- retrieve current external information
- combine these sources into an LLM request
- expose the AI capability through a REST API
- observe an LLM interaction using Phoenix
- package the application as a Docker image
- deploy and test the service on local Kubernetes

# 4. The Real Engineering Problem

An LLM is good at reasoning over information placed in its context. But your application owns information that the model does not automatically know.

For this scenario, three kinds of information live in three different places:

- Structured data: the client portfolio and risk tolerance live in PostgreSQL.
- Unstructured data: company trading rules live in a local policy PDF.
- Live external data: current stock news must be retrieved from the web.

Your backend therefore has an important responsibility: collect the relevant context before calling the LLM.

**Mental model: Think of your FastAPI application as the engineer who prepares the briefing pack for an analyst. The analyst can reason much better when the right information is assembled first.**

## 5. Context Injection - The Core Concept

The application gathers the three context sources and places them into clearly separated sections of the system prompt before sending the user question to the LLM.

The goal is not to make the LLM magically know the company. The goal is to give it the information it needs for this particular request.

## 6. Think Before You Code

Before opening your editor, take 10–15 minutes to reason about this scenario:

**A client with ID 1042 asks: “I want to buy \$15,000 worth of TSLA stock today. Given my current portfolio and your internal risk policies, is this allowed? Also, is there any bad news about TSLA today?”**

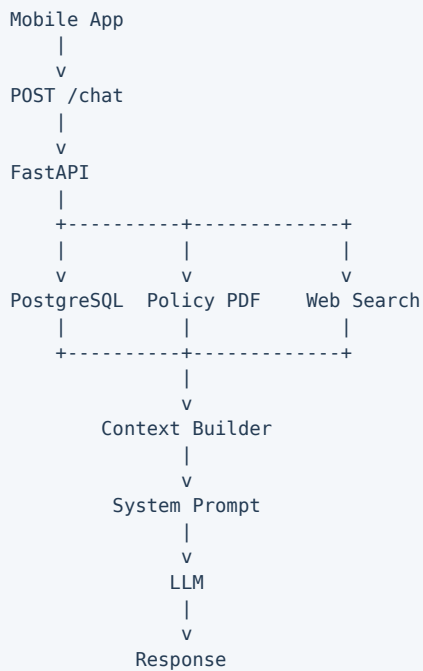
Answer these questions in your README or notebook before implementing:

1. What information does the LLM need?
2. Where does each piece of information come from?
3. Which part of the application should retrieve each piece?
4. When should that information be retrieved?
5. How will the three sources be clearly separated before they reach the LLM?
6. What could happen if one source is unavailable?
7. How will you test that the correct client data was retrieved?
8. How will you verify what was actually sent to the LLM?

Don't worry about writing the perfect design. The purpose is to make your thinking visible before you code.

## 7. Architecture

TEXT



Your job is to build these pieces one at a time and prove that each piece works before moving to the next.

## 8. Project Structure

Before writing the application, decide where each responsibility belongs.

We do not want one large `main.py` containing API code, database queries, PDF processing, web search, prompt construction, and LLM calls. Even in a small application, separating responsibilities makes the system easier to understand, test, and change.

For this assignment, use the following structure:

TEXT



```

financial-advisor-microservice/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── schema.py
│   ├── llm.py
│   ├── services/
│   │   ├── __init__.py
│   │   └── context_service.py
│   └── clients/
│       ├── __init__.py
│       ├── postgres_client.py
│       └── web_search_client.py
├── tests/
│   └── ...
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── .env.example
├── .gitignore
├── requirements.txt
├── Dockerfile
├── README.md
└── Apex_Financial_Risk_Policy_2026.pdf

```

## What belongs where?

| Component                        | Responsibility                                                        |
|----------------------------------|-----------------------------------------------------------------------|
| app/main.py                      | FastAPI routes and request/response flow                              |
| app/schema.py                    | Pydantic request and response models                                  |
| app/llm.py                       | LLM client and LLM interaction                                        |
| app/services/context_service.py  | Orchestrates collection of the context required by the AI application |
| app/clients/postgres_client.py   | PostgreSQL connection and database queries                            |
| app/clients/web_search_client.py | Web-search interaction                                                |
| tests/                           | Tests for application components                                      |
| k8s/                             | Kubernetes deployment configuration                                   |
| Dockerfile                       | Container image definition                                            |
| README.md                        | Setup, architecture, testing evidence, and reflection                 |

The policy PDF is a local application resource, so you do not need a separate client for it at this stage. The document-reading logic can be kept within the context service or separated later if it grows.

**Engineering principle: Do not create files just because a structure looks professional. Create a component when it has a clear responsibility.**

# Before You Code

Look at the architecture you designed in the previous section and think about this flow:



You should be able to explain why each component exists before you start implementing it.

## 9. Project Deliverables

Your final repository should contain:

- `app/main.py`
- `app/schema.py`
- `app/llm.py`
- `app/services/context_service.py`
- `app/clients/postgres_client.py`
- `app/clients/web_search_client.py`
- `requirements.txt`
- `Dockerfile`
- `k8s/deployment.yaml`
- `k8s/service.yaml`
- `README.md`
- `tests/` containing the tests you create
- `Apex_Financial_Risk_Policy_2026.pdf`

The exact functions inside these files are part of your design work. Use the responsibilities above as your guide.

## 9. Environment Setup

You will be provided with the database generator and the policy document.

1. Run the provided `generate_financial_db.py` script. It creates a local PostgreSQL database named `apex_financial` with mock clients and portfolios.
2. Place `Apex_Financial_Risk_Policy_2026.pdf` in your working directory. The document is around 200 pages.
3. Start your local Arize Phoenix server so traces can be captured.
4. Create and activate your Python virtual environment.
5. Create your Git repository and commit your work in small, meaningful steps.

**Don't be intimidated by the 200-page document. You are not expected to send 200 pages to the LLM. The engineering task is to narrow the document down to the relevant information before constructing the prompt.**

Suggested setup commands:

BASH



```
python -m venv .venv

# macOS / Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1

python --version
pip --version
```

## 10. Implementation - Guided Discovery

Do not try to write the entire application at once. Build it layer by layer. After each layer, run a small test and confirm what happened.

### Layer 1 - Data Extraction

Your first job is not to call the LLM. Your first job is to prove that you can collect the right information.

You need three retrieval responsibilities:

- retrieve the client portfolio information
- retrieve the relevant policy information
- retrieve current stock news

A useful design is to isolate these responsibilities into small helper functions. The exact implementation is yours, but the responsibilities should remain clear.

PYTHON



```
def get_client_portfolio(client_id: int) -> str:
    """Retrieve the client's relevant portfolio context."""
    pass

def get_policy_rules(risk_tolerance: str) -> str:
    """Extract policy information relevant to the client's risk profile."""
    pass

def get_stock_news(ticker: str) -> str:
    """Retrieve a small set of current news headlines."""
    pass
```

These signatures are guidance, not a complete solution.

## PostgreSQL - What should you think about?

- Which tables contain client information?
- Which table contains portfolio information?
- What relationship connects the client to the portfolio?
- What fields are actually needed by the LLM?
- What should happen when the client ID does not exist?

Hint: Keep the returned context small and meaningful. The LLM does not need your entire database row.

## The 200-page policy PDF - a practical first approach

Start simple. First understand how to read text from the document. Then identify a straightforward way to locate the policy information relevant to the client's risk tolerance.

PYTHON



```
from PyPDF2 import PdfReader

reader = PdfReader("Apex_Financial_Risk_Policy_2026.pdf")

for page in reader.pages:
    text = page.extract_text()
    # inspect the extracted text before deciding how to filter it
```

A sensible first approach is to scan the extracted text and keep only sections that appear relevant. You are not expected to build semantic search in this assignment.

**Engineering question: How can you reduce a large document into useful context before calling the LLM?**

## 11. The First Solution Is Not the Final Solution

Suppose your first approach searches for the exact risk tolerance string. It may work when the document contains the word "Conservative".

But what if the policy says “Clients who are risk-averse” instead?

**Pause and observe: Your retrieval logic may miss the relevant rule even though the meaning is related. That is an important limitation to notice. In the next session, we will address this kind of semantic retrieval problem with Vector Databases.**

For this assignment, keep the retrieval approach intentionally simple and understand its limitation.

## 12. Layer 2 - FastAPI and Observability

Once your retrieval functions work independently, expose them through a FastAPI service.

Your request model should contain the information the backend needs:

PYTHON



```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    client_id: int
    ticker: str
    user_query: str
```

Configure Phoenix instrumentation so that you can inspect the LLM interaction. Use the current official documentation for the library versions in your environment rather than blindly copying an old setup.

Ask yourself:

**“If the application gives an unexpected answer, how will I know what the LLM actually received?”**

## 13. Layer 3 - Context Injection

Inside POST /chat, your application should:

1. receive the client ID, ticker, and user question
2. retrieve the client portfolio context
3. retrieve the relevant policy context
4. retrieve current stock news
5. construct a clear system prompt with separate sections for each context source
6. send the system prompt and user question to the LLM
7. return the model response through FastAPI

A useful prompt structure is:

TEXT



SYSTEM

You are an assistant for Apex Financial Partners.

CLIENT PORTFOLIO

<retrieved portfolio context>

RISK POLICY

<retrieved policy context>

CURRENT STOCK NEWS

<retrieved news context>

Use the provided context when answering the user's question.

USER

<user question>

The important requirement is the separation of context and the reasoning behind it. Do not treat the example wording above as the complete solution.

**Important: Context injection improves the information available to the model. It does not guarantee a correct or “perfect” answer. Your job is to make the information flow explicit, testable, and observable.**

## 14. Layer 4 - Docker

Once the application works locally, package it so the same application can run in a container.

Your Dockerfile should:

- start from a suitable Python base image
- install dependencies
- copy the application into the image
- expose port 8000
- start the FastAPI application

Do not place your real API key directly into your source code or Dockerfile.

## 15. Layer 5 - Kubernetes

Your final application should run locally on Kubernetes using minikube or kind.

Create two manifests:

- Deployment - describes how the application container should run.
- Service - exposes the application so you can send requests to it.

Your Deployment should receive `GR0Q_API_KEY` through an environment variable. For a more production-ready design, use a Kubernetes Secret rather than placing the key directly in the manifest.

Typical commands you may need:

BASH



```
docker build -t financial-advisor:latest .

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

kubectl get pods
kubectl get services
```

The exact command to access the service depends on your local Kubernetes setup. Record your working command in README.md.

## 16. Testing the Complete Flow

Once Kubernetes is running, test the complete request from outside the application process.

BASH



```
curl -X POST http://localhost:<K8S_PORT>/chat \
-H "Content-Type: application/json" \
-d '{
  "client_id": 1042,
  "ticker": "TSLA",
  "user_query": "I want to buy $15,000 worth of TSLA stock today. Given my current portfolio and your internal risk policies"
}'
```

Replace <K8S\_PORT> with the port exposed by your Kubernetes Service.

## 17. Final Validation

Do not stop when you get a response. Validate the whole system.

| Area          | What to verify                                                                   |
|---------------|----------------------------------------------------------------------------------|
| API           | POST /chat returns a successful response for a valid request.                    |
| Database      | The portfolio belongs to the requested client.                                   |
| Policy        | Relevant policy information is retrieved instead of sending the entire document. |
| News          | A small set of current headlines is retrieved for the ticker.                    |
| Prompt        | Portfolio, policy, and news are clearly separated in the LLM context.            |
| LLM           | The response uses supplied context rather than inventing client-specific facts.  |
| Phoenix       | You can inspect the LLM trace and prompt/context.                                |
| Docker        | The application runs from your Docker image.                                     |
| Kubernetes    | The application is reachable through the Kubernetes Service.                     |
| Failure cases | You have tested at least one invalid or unavailable data source.                 |

## 18. Evidence to Include in README.md

Include:

- a simple architecture diagram or ASCII flow
- a short explanation of where each context source comes from
- evidence of a successful API call
- evidence that the Docker image runs
- evidence that Kubernetes Deployment and Service are running
- a Phoenix trace showing the LLM interaction
- one failure or edge case and how you handled it
- a short reflection: what worked, what was difficult, and what you would improve next

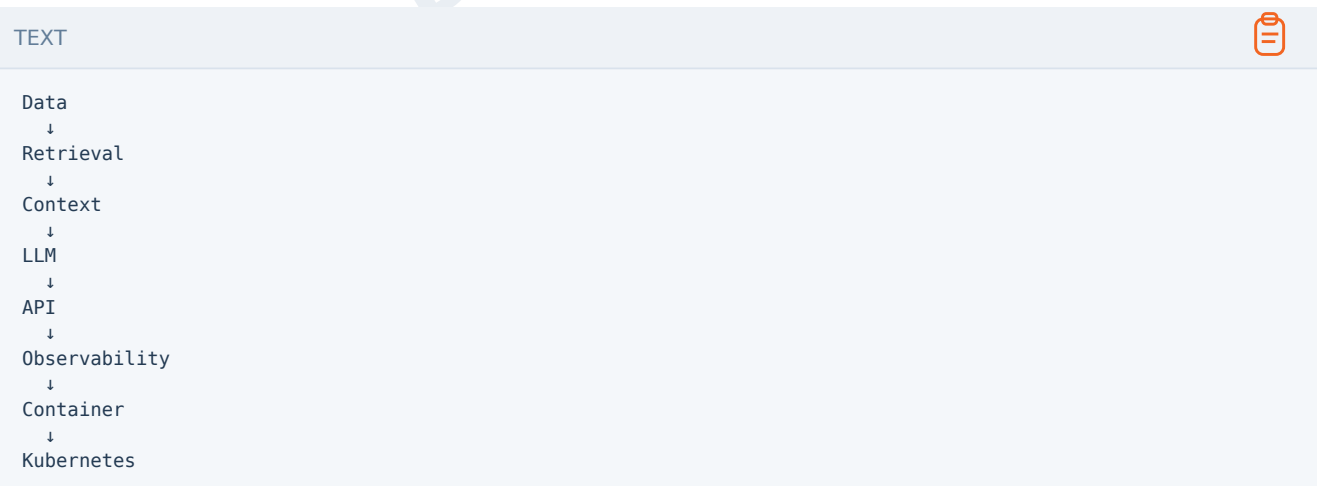
## 19. Engineering Reflection

Answer these questions in your README. Keep the answers concise, but explain your reasoning.

1. Why is context retrieval the responsibility of the application rather than something you expect the LLM to do automatically?
2. Why is it a bad idea to send the entire 200-page policy document to the model?
3. What is the weakness of exact keyword matching in document retrieval?
4. Why is observability important in an AI application?
5. What information would you want to see in a trace when debugging a wrong answer?
6. What would you change if this application had thousands of users?
7. What part of this design would you replace when we introduce vector databases?

## 20. What You Have Built

If you completed the assignment, you have built more than an LLM call.



This is the beginning of thinking about AI applications as engineered systems rather than isolated model calls.

**Next step: Today your retrieval approach is intentionally simple. In the next stage, we will address the limitation you discovered and explore semantic retrieval with Vector Databases.**

**“Don’t focus on getting the answer. Focus on learning how to engineer the solution.”**

Uma Kiran | DataKern | [www.data-kern.com](http://www.data-kern.com)

DATAKERN